

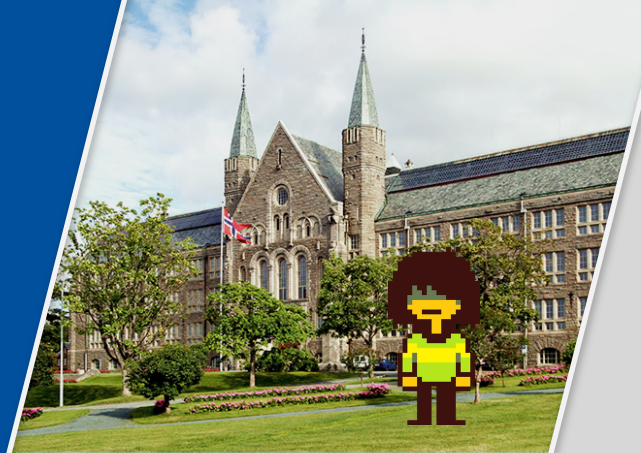


Norwegian University of
Science and Technology

Assignment Lecture 2

Constraint Satisfaction Problems

TDT4137 – Introduction to Artificial Intelligence, 2026
Aleksandra Jekic, Department of Computer Science



Relevant for this assignment



- Lecture 4 (yesterday, 10.09.26)
- Chapter 5 in the Russell and Norvig book (4th edition, Global).

How important is this?

- Historically, there has usually been a problem covering this in exams.
- Not the hardest topic and the algorithms are quite intuitive. However, if you are not familiar with the terminology the problem will seem like gibberish.
- Watch out, read the book! Last year there was a problem on topological sort. I believe this was only in the book and a lot of people were confused.
- Note: Assignment is programming only, which is different from exams. Might be a good idea to look at some old exam problems now as well.

Constraint satis-what?

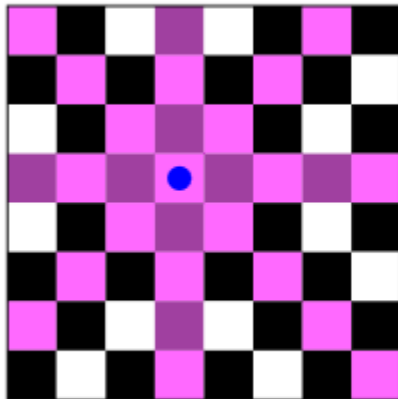
- What are we doing now: You want to decide the value of lots of variables given a bunch of possible values and certain constraints.
- A lot of problems have this underlying structure such as scheduling different teachers for lectures, what order to build a car in given limited resources, deciding where to seat people at a dinner party who have very strong opinions about each other, solving Sudoku boards, etc.
- This is an NP-complete problem. Can make a lot of money for companies who do things at a large scale by improving their schedules.

Constraint satis-what?

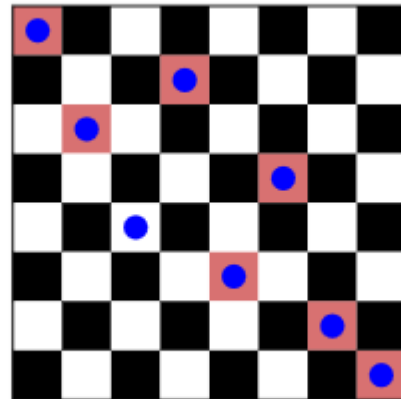
- You need a set of variables, $X = \{\text{seat1}, \text{seat2}, \dots, \text{seat7}\}$.
- You need a domain for each variable, $D_{\text{seat1}} = \{\text{Emily}, \text{Anne}, \dots, \text{Charlotte}\}$.
- You need constraints like $C(\text{Emily}, \text{Charlotte})$: Not adjacent, etc.
- Doesn't really matter how you represent this.
- Intuition: Probably some of these constraints immediately rule out possibilities (constraint propagation like AC-3). Then we can just try some assignments and go back if we get stuck (backtrack search). We can also update connected domains after making each move (forward checking).

Small problem

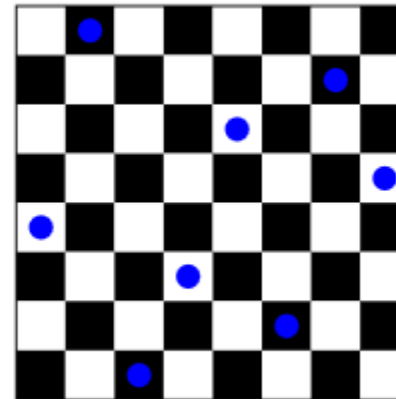
Place 8 queens in a chess board such that no queen checks each other.



(a) Queen constraints



(b) Generated conflicts



(c) Possible solution

- The 8-queens problem was used as an example of a problem that is appropriate for genetic algorithms in a recent lecture.
- Would genetic algorithms or a CSP solvers be more efficient? Why?

Assignment 2

- Basically, you're going to run AC-3 on the constraint network (variables, domains and constraints) of several Sudoku boards.
- Pseudocode for this constraint propagation algorithm is found on p. 171 in Norvig & Russell's book (4th edition, Global).
- Afterwards you run backtracking-search, pseudocode found on p. 176 in Norvig & Russell's book (4th edition, Global).
- AC-3 may immediately complete some of the simpler boards and that is fine!

							1	
4								
	2							
				5		4		7
		8				3		
		1		9				
3			4			2		
	5		1					
			8		6			

Assignment 2

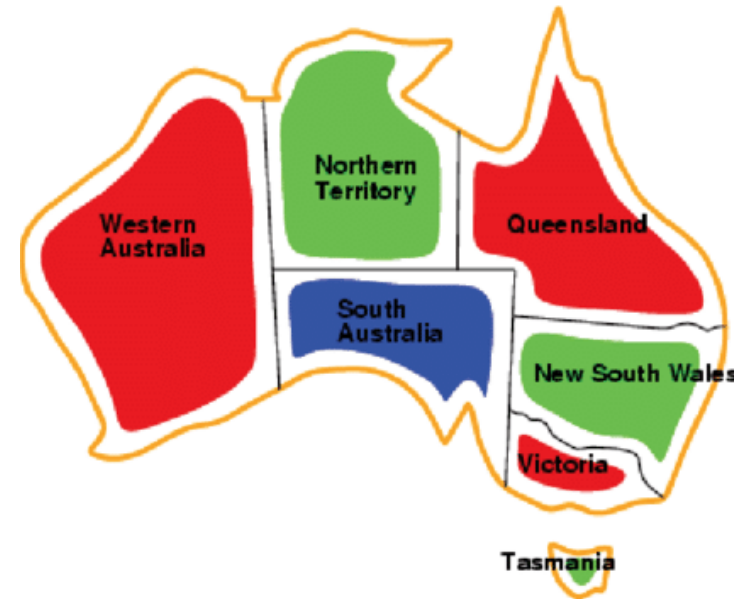
- You are very lucky!
- We have already programmed almost everything for you in Python.
- The CSP class, visualizing the Sudoku boards, etc.
- We have also provided the simple map coloring problem that you can easily use to check if your algorithm does its job.
- You just need to complete the code for **AC-3** and **backtrack-search**.
- Of course, if you want to – you can represent the problem however you want in whatever language you would like to from scratch.

Assignment 2

```
from csp import CSP, alldiff

variables = ['WA', 'NT', 'Q', 'NSW', 'V', 'SA', 'T']
csp = CSP(
    variables=variables,
    domains={variable: {'red', 'green', 'blue'}
             for variable in variables},
    edges=[
        ('SA', 'WA'),
        ('SA', 'NT'),
        ('SA', 'Q'),
        ('SA', 'NSW'),
        ('SA', 'V'),
        ('WA', 'NT'),
        ('NT', 'Q'),
        ('Q', 'NSW'),
        ('NSW', 'V'),
    ],
)

print(csp.backtracking_search())
```



- You have the files `csp.py`, `map_coloring.py` and `sudoku.py`.

Assignment 2

```
from typing import Any
from queue import Queue

class CSP:
    def __init__(
        self,
        variables: list[str],
        domains: dict[str, set],
        edges: list[tuple[str, str]],
    ):
        """Constructs a CSP instance with the given variables, domains and edges.

        Parameters
        -----
        variables : list[str]
            The variables for the CSP
        domains : dict[str, set]
            The domains of the variables
        edges : list[tuple[str, str]]
            Pairs of variables that must not be assigned the same value
        """
        self.variables = variables
        self.domains = domains
```

- From csp.py

Assignment 2

```
# Binary constraints as a dictionary mapping variable pairs to a set of value pairs.
#
# To check if variable1=value1, variable2=value2 is in violation of a binary constraint:
# if (
#     (variable1, variable2) in self.binary_constraints and
#     (value1, value2) not in self.binary_constraints[(variable1, variable2)]
# ) or (
#     (variable2, variable1) in self.binary_constraints and
#     (value1, value2) not in self.binary_constraints[(variable2, variable1)]
# ):
#     Violates a binary constraint
self.binary_constraints: dict[tuple[str, str], set] = {}
for variable1, variable2 in edges:
    self.binary_constraints[(variable1, variable2)] = set()
    for value1 in self.domains[variable1]:
        for value2 in self.domains[variable2]:
            if value1 != value2:
                self.binary_constraints[(variable1, variable2)].add((value1, value2))
                self.binary_constraints[(variable1, variable2)].add((value2, value1))
```

- From csp.py

Assignment 2

```
def backtracking_search(self) -> None | dict[str, Any]:
    """Performs backtracking search on the CSP.

    Returns
    -----
    None | dict[str, Any]
    | A solution if any exists, otherwise None
    """
    def backtrack(assignment: dict[str, Any]):
        # YOUR CODE HERE (and remove the assertion below)
        assert False, "Not implemented"

    return backtrack({})
```

- From csp.py

Example output after implementing `csp.backtracking_search()`:
{ 'WA': 'red', 'NT': 'green', 'Q': 'red', 'NSW': 'green', 'V': 'red', 'SA': 'blue', 'T': 'red' }

Assignment 2

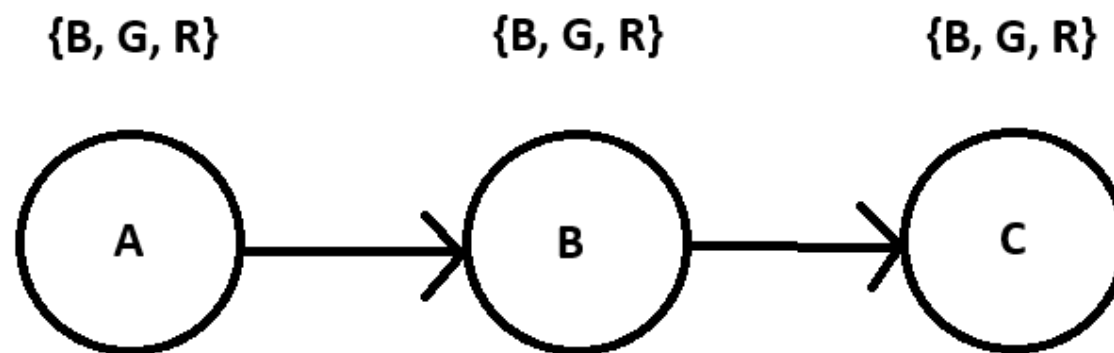
- Select-Unassigned-Variable?
 - Anything is fine. Can try minimum-remaining-values (MRV) heuristic if you want.
- Order-Domain-Values?
 - Anything is fine. Can try least-constraining-value (LCV) heuristic if you want.
- Inference?
 - This is where you would do forward-check. Just skip.

```
function BACKTRACKING-SEARCH(csp) returns a solution or failure
    return BACKTRACK(csp, {})

function BACKTRACK(csp, assignment) returns a solution or failure
    if assignment is complete then return assignment
    var ← SELECT-UNASSIGNED-VARIABLE(csp, assignment)
    for each value in ORDER-DOMAIN-VALUES(csp, var, assignment) do
        if value is consistent with assignment then
            add {var = value} to assignment
            inferences ← INFERENCE(csp, var, assignment)
            if inferences ≠ failure then
                add inferences to csp
                result ← BACKTRACK(csp, assignment)
                if result ≠ failure then return result
            remove inferences from csp
            remove {var = value} from assignment
    return failure
```

Small problem

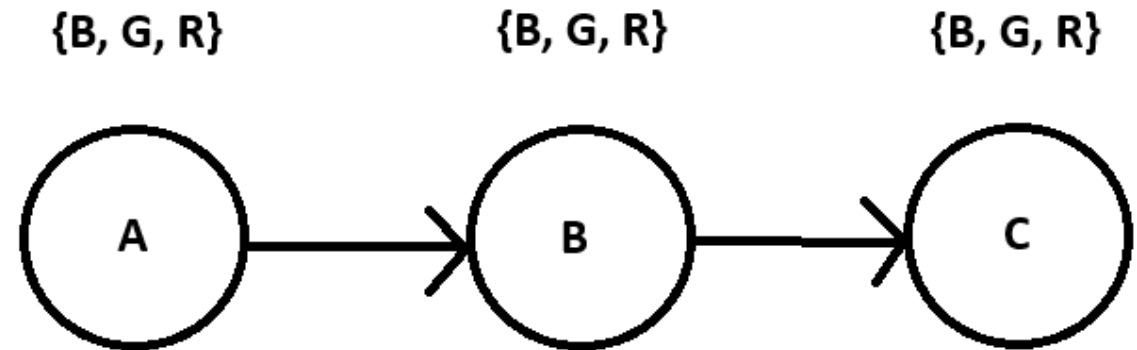
- Very simple constraint problem.
- Assume minimum-remaining-value heuristic for variables and least-constraining-value heuristic for value assignments.
- Break ties alphabetically.
- Is MRV+LCV better than random choices on average for the graph on the right?



- Don't worry about exactly calculating expected mistakes.

Small problem

- Basically with the heuristics you are **guaranteed** 1 mistake.
- With random choices, 4/6 times you will pick B first or second which gives you $1/3 + 1/3 = 2/3$ expected mistakes.
- The two other cases are more tedious, but the expected amount of mistakes is $(4(2/3) + 2(7/9))/6 = \sim 0.704$.
- $1/3$ (A and C, same color) times $1/3$ (expected mistakes) $= 1/9$. $2/3$ (A and C, different color) times 1 (expected mistakes) $= 2/3$. Now $1/9 + 2/3 = 7/9$.



Assignment 2

```
# Choose Sudoku problem
grid = open('sudoku_easy.txt').read().split()

width = 9
box_width = 3

domains = {}
for row in range(width):
    for col in range(width):
        if grid[row][col] == '0':
            domains[f'X{row+1}{col+1}'] = set(range(1, 10))
        else:
            domains[f'X{row+1}{col+1}'] = {int(grid[row][col])}

edges = []
for row in range(width):
    edges += alldiff([f'X{row+1}{col+1}' for col in range(width)])
for col in range(width):
    edges += alldiff([f'X{row+1}{col+1}' for row in range(width)])
for box_row in range(box_width):
    for box_col in range(box_width):
        cells = []
        edges += alldiff(
            [
                f'X{row+1}{col+1}' for row in range(box_row * box_width, (box_row + 1) * box_width)
                for col in range(box_col * box_width, (box_col + 1) * box_width)
            ]
        )
```

- From sudoku.py

							1	
4								
	2							
				5		4		7
		8				3		
		1		9				
3			4			2		
	5		1					
			8		6			

Assignment 2

```
def alldiff(variables: list[str]) -> list[tuple[str, str]]:
    """Returns a list of edges interconnecting all of the input variables

    Parameters
    -----
    variables : list[str]
    |   The variables that all must be different

    Returns
    -----
    list[tuple[str, str]]
    |   List of edges in the form (a, b)
    """
    return [(variables[i], variables[j]) for i in range(len(variables) - 1) for j in range(i + 1, len(variables))]
```

- From csp.py

Assignment 2

```
✓ csp = CSP(  
    variables=[f'X{row+1}{col+1}' for row in range(width) for col in range(width)],  
    domains=domains,  
    edges=edges,  
)  
  
print(csp.ac_3())  
print_solution(csp.backtracking_search())
```

- From sudoku.py

Assignment 2

```
def print_solution(solution):
    """
    Convert the representation of a Sudoku solution, as returned from
    the method CSP.backtracking_search(), into a Sudoku board.
    """
    for row in range(width):
        for col in range(width):
            print(solution[f'X{row+1}{col+1}'], end=" ")
            if col == 2 or col == 5:
                print('|', end=" ")
        print("")
        if row == 2 or row == 5:
            print('-----+-----+-----')
```

```
# 7 8 4 | 9 3 2 | 1 5 6
# 6 1 9 | 4 8 5 | 3 2 7
# 2 3 5 | 1 7 6 | 4 8 9
# -----+-----+-----
# 5 7 8 | 2 6 1 | 9 3 4
# 3 4 1 | 8 9 7 | 5 6 2
# 9 2 6 | 5 4 3 | 8 7 1
# -----+-----+-----
# 4 5 3 | 7 2 9 | 6 1 8
# 8 6 2 | 3 1 4 | 7 9 5
# 1 9 7 | 6 5 8 | 2 4 3
```

- From sudoku.py

Assignment 2

```
def ac_3(self) -> bool:
    """Performs AC-3 on the CSP.
    Meant to be run prior to calling backtracking_search() to reduce the search for some problems.

    Returns
    -----
    bool
    |   False if a domain becomes empty, otherwise True
    """
    # YOUR CODE HERE (and remove the assertion below)
    assert False, "Not implemented"
```

- From csp.py

Assignment 2

```
function AC-3(csp) returns false if an inconsistency is found and true otherwise  
  queue  $\leftarrow$  a queue of arcs, initially all the arcs in csp
```

```
  while queue is not empty do  
    (Xi, Xj)  $\leftarrow$  POP(queue)  
    if REVISE(csp, Xi, Xj) then  
      if size of Di = 0 then return false  
      for each Xk in Xi.NEIGHBORS - {Xj} do  
        add (Xk, Xi) to queue  
  return true
```

```
function REVISE(csp, Xi, Xj) returns true iff we revise the domain of Xi  
  revised  $\leftarrow$  false  
  for each x in Di do  
    if no value y in Dj allows (x,y) to satisfy the constraint between Xi and Xj then  
      delete x from Di  
      revised  $\leftarrow$  true  
  return revised
```

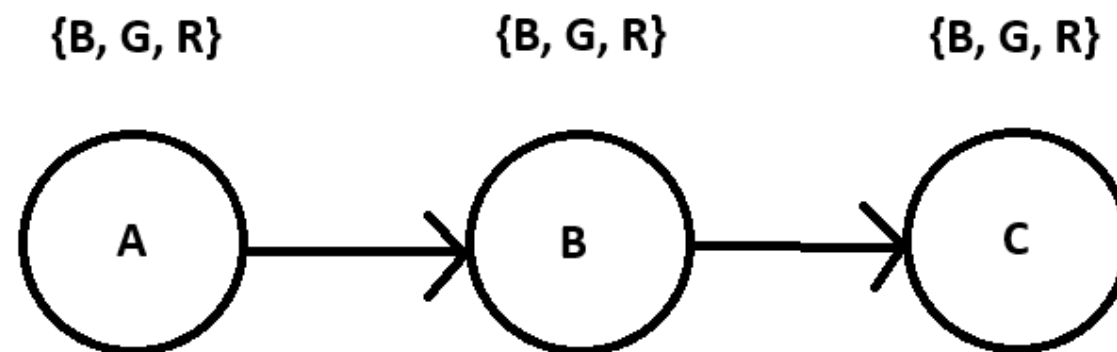
Assignment 2

4 Deliverables

1. Source files containing well-commented code for backtracking search and AC-3 that are able to solve the Sudoku boards.
2. A **single** PDF report containing:
 - (a) Your program's solution for each of the Sudoku boards shown in Figures 1a-1d.
 - (b) The domains after running the AC-3 algorithm for each of the four boards.
 - (c) The number of times your `backtrack()` function was *called*, and the number of times your `backtrack()` function returned failure, for each of the four boards.
 - (d) The runtime of the backtracking search algorithm for each of the four boards.
 - (e) The total runtime of the AC-3 and backtracking search algorithms for each of the four boards.
 - (f) Brief discussion of your results. For example, explain why the AC-3 algorithm drastically decrease the total runtime when solving some Sudoku problems.

Small problem

- Going back to our old friend.
- Assume we make A blue.
- What do the domains look like after AC-3?
- B's domain becomes {G, R}.
- Is MRV+LCV better than random choices now? Break ties alphabetically.



Practical reminders

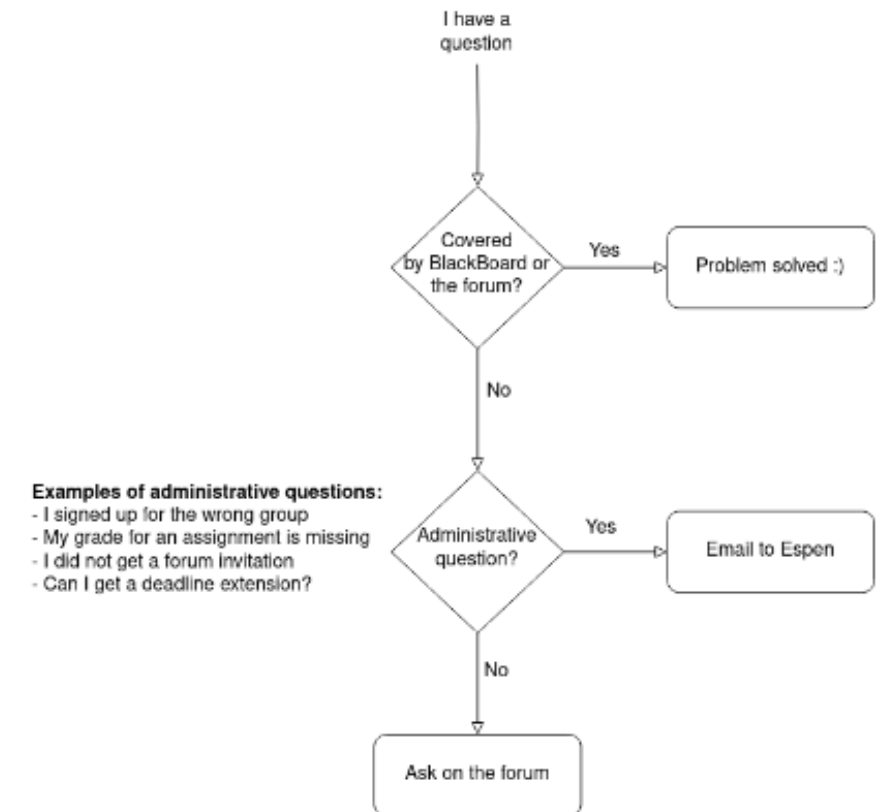
Recommendations

Make sure that your code is well documented. **Please upload the code files and PDF report as separate files when delivering the assignment on Canvas. Do NOT put in zip.**

During lab hour periods, lab hours will be arranged:

- Monday - 16:15-18:00. Place: [GL-AO U35](#) ➞
- Tuesday at 14:15 to 16:00. Place: [GL-AO U36](#) ➞
- Wednesday at 08:15 to 10:00. Place: [GL-AO U35](#) ➞
- Fridays at 14:15 to 16:00. Place: [GL-AO U36](#) ➞

Any questions or comments?





For more
information:
www.ntnu.edu

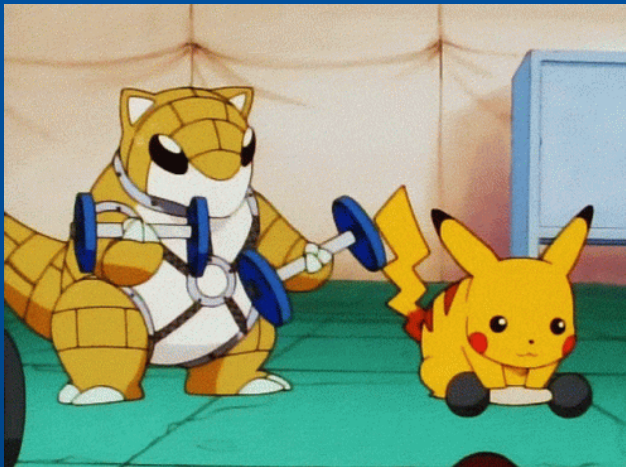


Photo – first slide and this slide: Gløshaugen – Photo: Mentz Indergaard/NTNU
Gjøvik – Photo: Espen Taftø Vestad/NTNU | Ålesund – Photo: Sissel Basso/NTNU

